

SMART VEHICLE RENTAL SYSTEM

IT23431 –MONGODB PROJECT REPORT

Submitted by

BALAMURUGAN R – 241001026

ASWIN S-241001024

OF

BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project titled “Smart Vehicle Rental System” is the bonafide work of “**BALAMURUGAN R (241001026), ASWIN S (241001024),**” who carried out the project work under my supervision.

SIGNATURE

Dr.Valarmathie

HEAD OF THE DEPARTMENT

Information Technology

Rajalakshmi Engineering College,

Rajalakshmi Nagar, Thandalam

Chennai – 602105

SIGNATURE

Mrs.Usha S

COURSE INCHARGE

Information Technology

Rajalakshmi Engineering College,

Rajalakshmi Nagar, Thandalam

Chennai - 602105

This project is submitted for IT23431 –MongoDB Essentials held on _____

INTERNAL EXAMINAR

EXTERNAL EXAMINAR

TABLE OF CONTENTS

S.No	Title	Page No
1.1	Abstract	4
1.2	Introduction	4
1.3	Purpose	4
1.4	Scope of the Project	5
1.5	Functional Specification	5
2.1	Use case Diagram	6
2.2	Entity-relationship Diagram	6
2.3	Data flow Diagram	7
3	Module Description	8
4	Screenshot	9
5	Code Implementation	12
6	Conclusion	22
7	References	22

SMART VEHICLE RENTAL SYSTEM

1.1 ABSTRACT

The Smart Vehicle Rental System is a web-based application designed to simplify and automate vehicle rental operations such as vehicle booking, customer management, and rental administration. The system uses HTML, CSS, and JavaScript as the front-end technologies and Python as the back-end with MongoDB as the database for secure and efficient data storage. Its main objective is to provide a centralized platform that minimizes manual effort, reduces errors, and improves operational efficiency in vehicle rental management.

The integration of MongoDB ensures flexible data handling, fast retrieval, and high scalability. With a user-friendly interface, responsive design, and database-driven architecture, this project demonstrates how modern web technologies can be effectively combined to create a robust, scalable, and efficient vehicle rental solution.

1.2 INTRODUCTION

The Smart Vehicle Rental System is a web-based application that uses MongoDB as its database and Python as the backend to automate and simplify vehicle rental operations. It manages key tasks such as vehicle booking, customer registration, and rental/return processes. The system reduces manual work, minimizes errors, and ensures efficient handling of customer information and vehicle availability.

With its responsive user interface built using HTML, CSS, and JavaScript, and secure backend processing using Python, it allows rental staff and users to manage activities more effectively. This project aims to enhance accuracy, speed, and reliability in vehicle rental services while providing a scalable and efficient solution for modern transportation needs.

1.3 PURPOSE

The purpose of the Smart Vehicle Rental System is to automate rental operations and improve overall management efficiency. It helps reduce manual paperwork, eliminate data redundancy, and ensure accurate handling of customer records, vehicle availability, and rental transactions. The system provides a digital platform for administrators, staff, and customers to manage rental activities quickly and efficiently.

1.4 SCOPE OF THE PROJECT

This project covers key rental functions including vehicle booking, customer registration, rental and return management, vehicle details, and billing. It can be implemented in small to medium-sized rental businesses to streamline daily operations, enhance data security, and provide a reliable foundation for future upgrades such as online payments, GPS tracking, and mobile app integration.

1.5 FUNCTIONAL REQUIREMENTS

1. Login Module (LM)

1. Users and admins log in with username and password.
2. Passwords are securely encrypted.
3. Access is granted after authentication using backend verification.

2. Vehicle Management Module (VMM)

1. Add, view, update, or delete vehicle details.
2. Manage vehicle types, pricing, and availability.

3. Booking Module (BM)

1. Users can search and book vehicles.
2. Real-time vehicle availability checking.
3. Modify or cancel bookings.

4. Rental/Return Module (RRM)

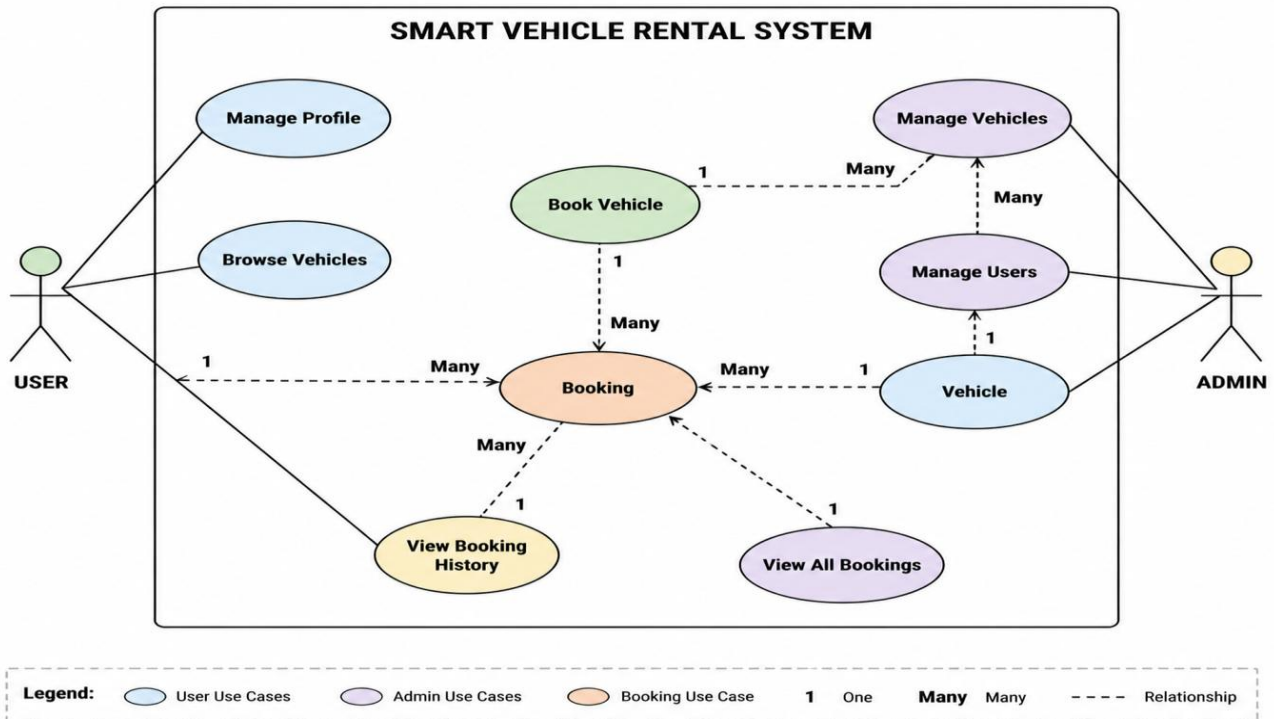
1. Manage vehicle pickup and return process.
2. Automatically update vehicle status.

5. Billing & Payment Module (BPM)

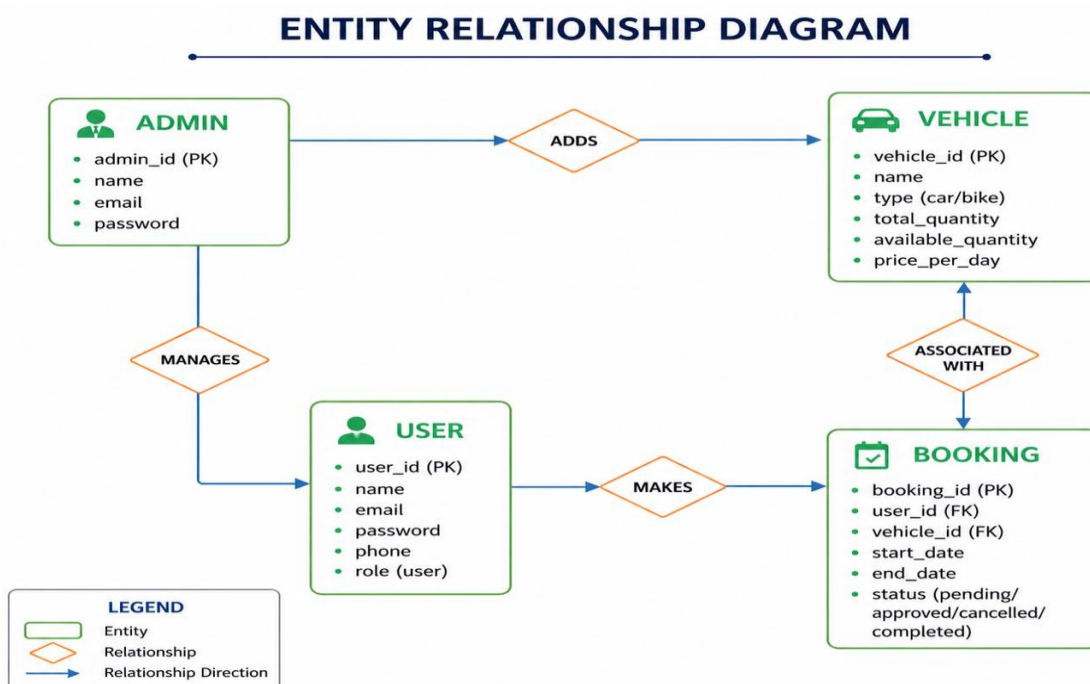
1. Calculate rental charges based on duration and vehicle type.
2. Generate invoices and payment receipts.
3. Support online payment integration (future scope).

2. SYSTEM FLOW DIAGRAM

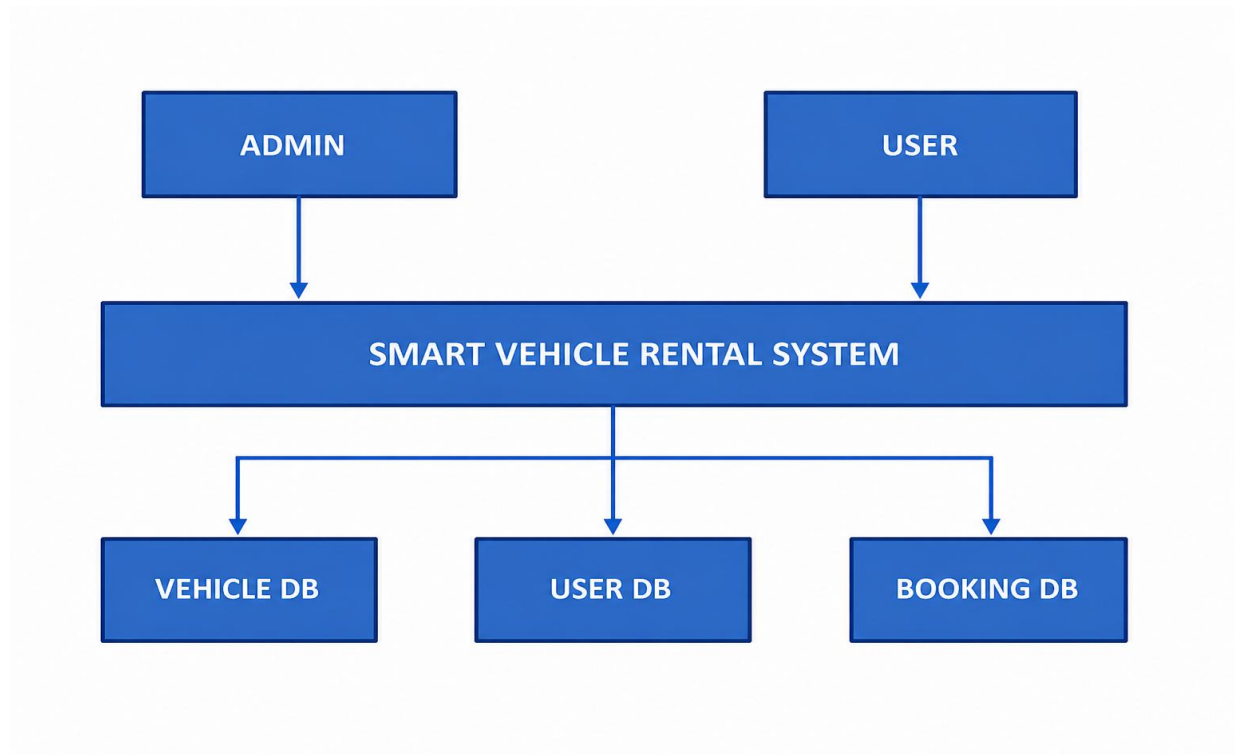
2.1 USE CASE DIAGRAM



2.1 ENTITY RELATIONSHIP DIAGRAM



2.3 DATA FLOW DIAGRAMS



3. MODULE DESCRIPTION

1. ADMIN LOGIN

Admin can log in using a valid username and password provided by the system.

2. ADD VEHICLES / MANAGE VEHICLES

Admin can add new vehicles, update vehicle details, and remove vehicles from the database.

3. MANAGE USERS

Admin can view, add, or remove users and manage their information in the system.

4. NEW USER REGISTRATION

New users can be added to the system by entering their details, which are stored in the user database.

5. SEARCH VEHICLES

Admin can search available and booked vehicles and check their availability status.

6. BOOKING MANAGEMENT (BOOK / CANCEL / COMPLETE)

Admin can manage bookings by approving, cancelling, or completing them. Booking details are stored and updated in the database.

7. VEHICLE AVAILABILITY UPDATE

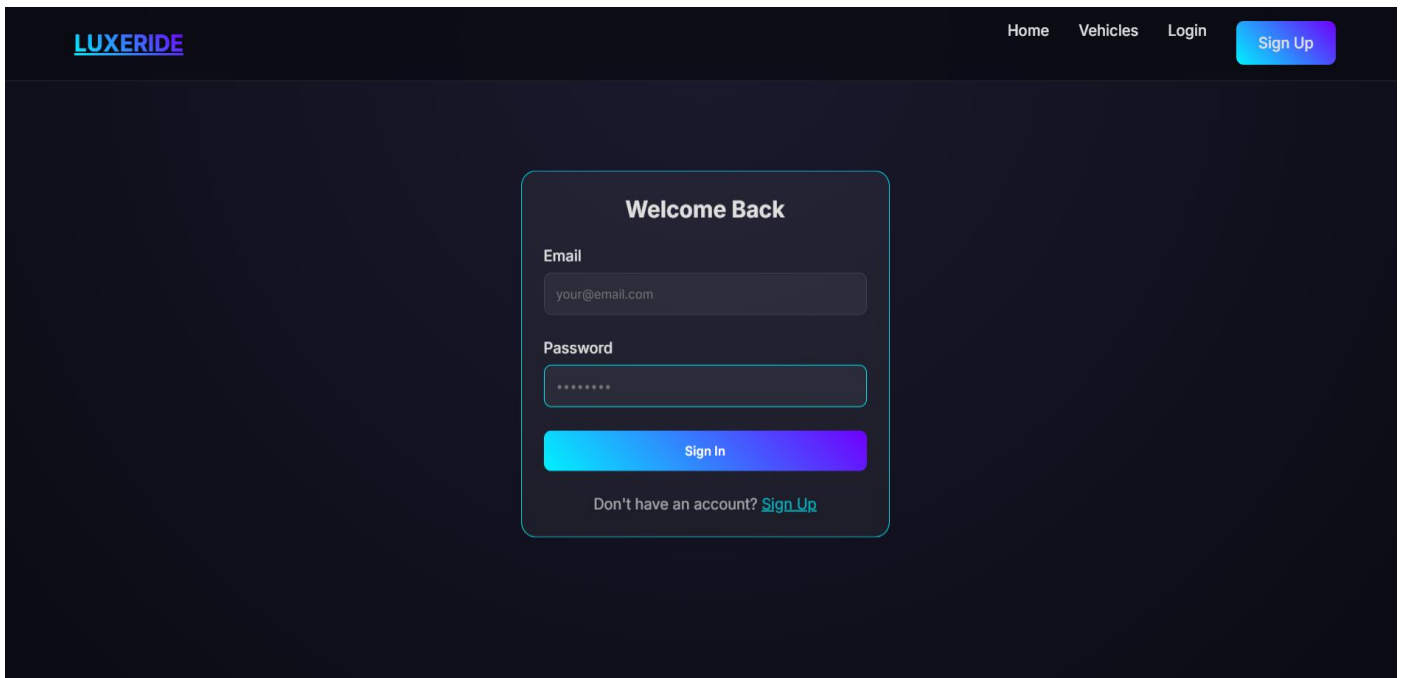
When a vehicle is booked, availability decreases, and when the booking ends, availability is updated automatically.

8. LOGOUT

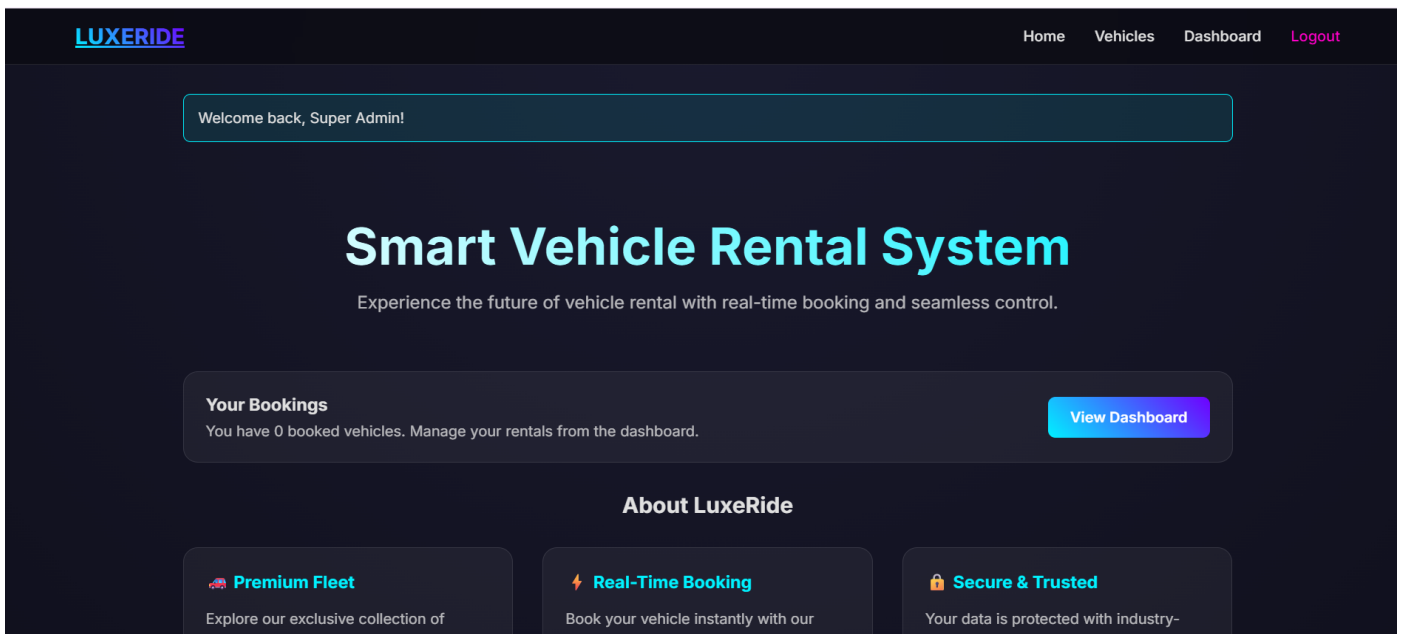
Admin can securely log out after completing all operations.

4. SCREENSHOT

LOGIN PAGE :



HOME PAGE:



EDIT VEHICLES PAGE:

LUXERIDE

HomeVehiclesDashboardLogout

Available Vehicles3 Vehicles



Tesla Model S

Car

Price: \$120/day

Available: 5

Vehicle Details

Type: Car
Daily Rate: \$120
Available Quantity: 5

Edit



Ducati Panigale V4

Bike

Price: \$80/day

Available: 7

Vehicle Details

Type: Bike
Daily Rate: \$80
Available Quantity: 7

Edit

ADMIN DASHBOARD:

LUXERIDE

HomeVehiclesDashboardLogout

Global Booking ManagementManage Fleet

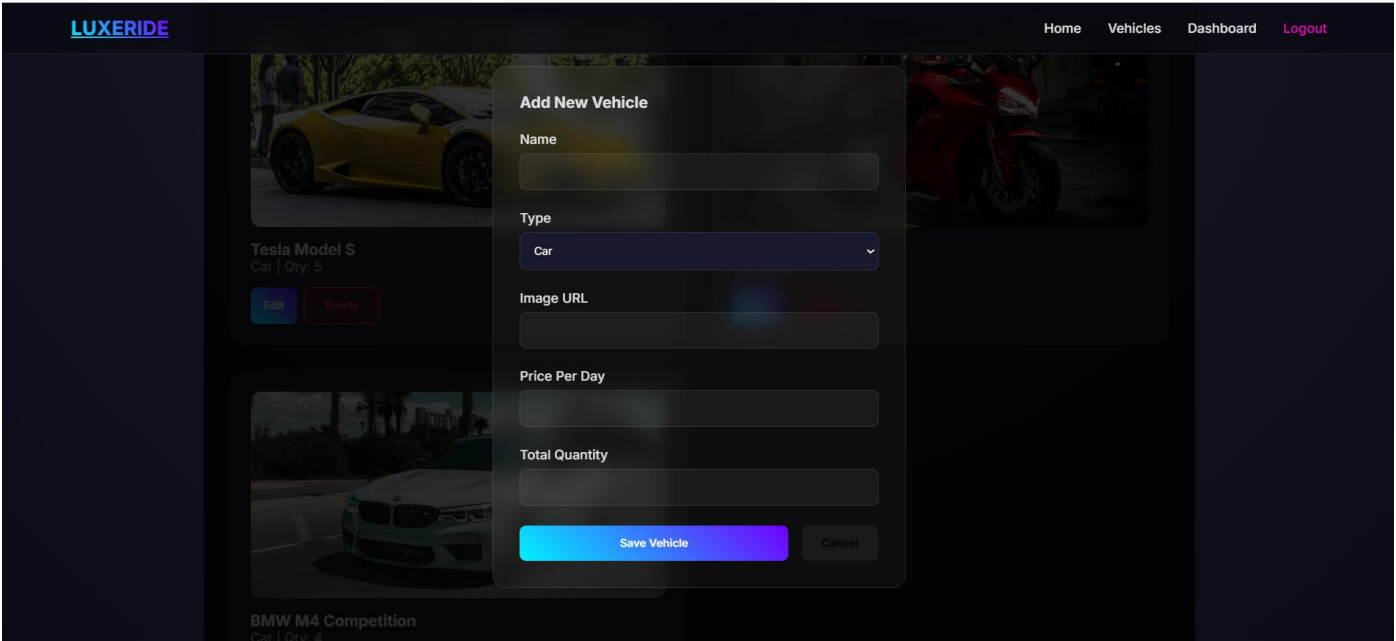
All Reservations

User	Vehicle	Dates	Status	Actions
arun arun@gmail.com	Ducati Panigale V4	04 May - 06 May 2026	Completed	

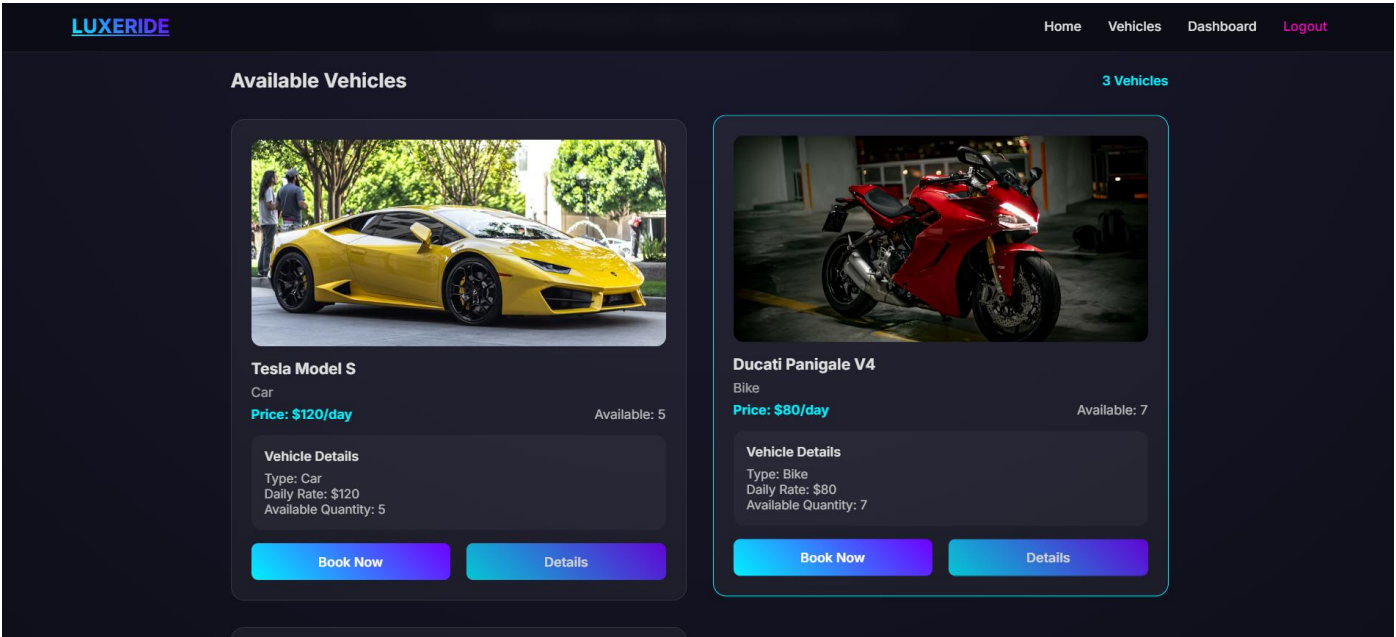
© 2024 LuxeRide Smart Vehicle Rental. All rights reserved.

10

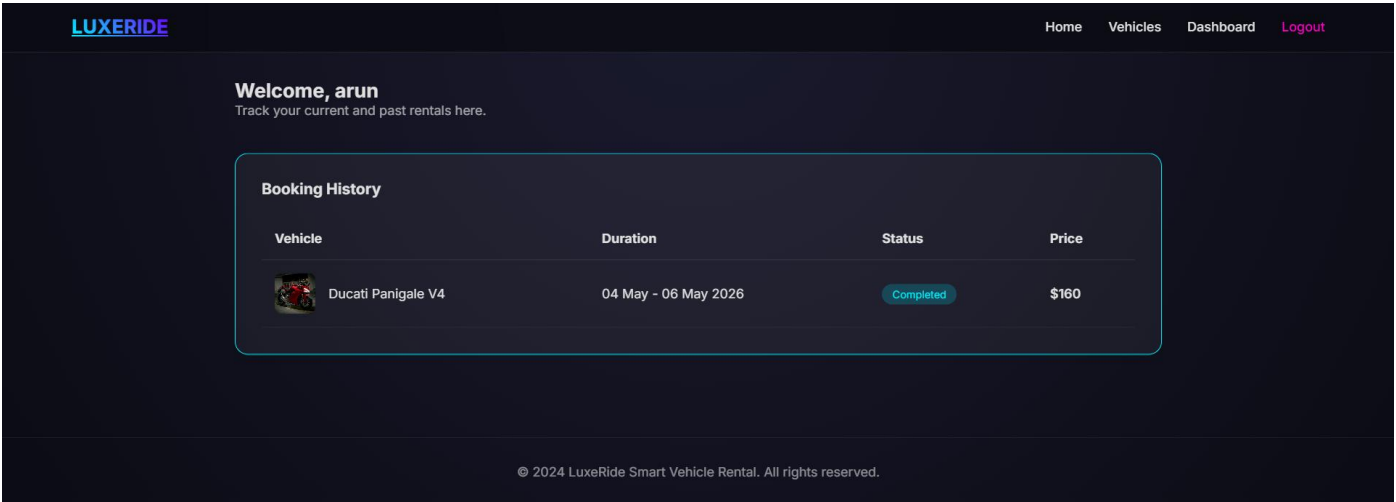
ADD VEHICLES PAGE:



BOOKING VEHICLES:



DASHBOARD:



5.CODE IMPLEMENTATION

```
from flask import Flask, render_template, request, redirect, url_for, flash, session
from flask_pymongo import PyMongo
from flask_bcrypt import Bcrypt
from bson.objectid import ObjectId
from datetime import datetime
from functools import wraps
from pymongo.errors import ServerSelectionTimeoutError
import os
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
app = Flask(__name__)
app.secret_key = os.environ.get('SECRET_KEY', 'supersecretkey')
```

```
# MongoDB Configuration
app.config["MONGO_URI"] = os.environ.get('MONGO_URI',
'mongodb://localhost:27017/vehicle_rental')
mongo = PyMongo(app)
bcrypt = Bcrypt(app)
```

```
@app.errorhandler(ServerSelectionTimeoutError)
def handle_mongo_timeout(error):
    message = (
        "Cannot connect to MongoDB. "
        "Please ensure the MongoDB server is running on localhost:27017 "
        "or update MONGO_URI in your .env file."
    )
    return render_template('error.html', message=message), 503
```

```
# Helper: Check if user is logged in
```

```

def is_logged_in():
    return 'user_id' in session

# Helper: Check if user is admin
def is_admin():
    return session.get('role') == 'admin'

# Helper: Protect routes that need authentication
def login_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        if not is_logged_in():
            return redirect(url_for('login', next=request.path))
        return f(*args, **kwargs)
    return decorated_function

@app.route('/')
@login_required
def index():
    vehicles = mongo.db.vehicles.find({"available_quantity": {"$gt": 0}})
    booking_count = 0
    if session.get('user_id'):
        booking_count = mongo.db.bookings.count_documents({"user_id": ObjectId(session['user_id'])})
    return render_template('index.html', vehicles=vehicles,
        booking_count=booking_count)

@app.route('/vehicles')
@login_required
def vehicles_page():
    vehicles = list(mongo.db.vehicles.find({"available_quantity": {"$gt": 0}}))
    booking_count = 0
    if session.get('user_id'):

```

```
        booking_count = mongo.db.bookings.count_documents({"user_id":
ObjectId(session['user_id'])})
    return render_template('vehicles.html', vehicles=vehicles,
booking_count=booking_count)
```

```
@app.route('/signup', methods=['GET', 'POST'])
def signup():
```

```
    if is_logged_in():
        return redirect(url_for('index'))
```

```
    if request.method == 'POST':
        name = request.form.get('name')
        email = request.form.get('email')
        password = request.form.get('password')
        role = 'user' # Default role
```

```
    # Check if email exists
```

```
    if mongo.db.users.find_one({"email": email}):
        flash("Email already exists", "danger")
        return redirect(url_for('signup'))
```

```
    hashed_password =
bcrypt.generate_password_hash(password).decode('utf-8')
    mongo.db.users.insert_one({
        "name": name,
        "email": email,
        "password": hashed_password,
        "role": role
    })
    flash("Signup successful! Please login.", "success")
    return redirect(url_for('login'))
    return render_template('signup.html')
```

```
@app.route('/login', methods=['GET', 'POST'])
```

```

def login():
    if is_logged_in():
        return redirect(url_for('index'))

    if request.method == 'POST':
        email = request.form.get('email')
        password = request.form.get('password')

        user = mongo.db.users.find_one({"email": email})
        if user and bcrypt.check_password_hash(user['password'], password):
            session['user_id'] = str(user['_id'])
            session['name'] = user['name']
            session['role'] = user['role']
            flash(f"Welcome back, {user['name']}!", "success")
            next_page = request.args.get('next') or url_for('index')
            return redirect(next_page)
        else:
            flash("Invalid email or password", "danger")
    return render_template('login.html')

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('index'))

@app.route('/dashboard')
def dashboard():
    if not is_logged_in():
        return redirect(url_for('login'))

    if is_admin():
        return redirect(url_for('admin_dashboard'))

    user_id = session['user_id']

```

```

bookings = list(mongo.db.bookings.aggregate([
    {"$match": {"user_id": ObjectId(user_id)}},
    {"$lookup": {
        "from": "vehicles",
        "localField": "vehicle_id",
        "foreignField": "_id",
        "as": "vehicle"
    }},
    {"$unwind": "$vehicle"}
]))
return render_template('dashboard.html', bookings=bookings)

```

```

@app.route('/admin_dashboard')
def admin_dashboard():
    if not is_admin():
        flash("Unauthorized access", "danger")
        return redirect(url_for('index'))

```

```

bookings = list(mongo.db.bookings.aggregate([
    {"$lookup": {
        "from": "users",
        "localField": "user_id",
        "foreignField": "_id",
        "as": "user"
    }},
    {"$unwind": "$user"},
    {"$lookup": {
        "from": "vehicles",
        "localField": "vehicle_id",
        "foreignField": "_id",
        "as": "vehicle"
    }},
    {"$unwind": "$vehicle"}
]))

```



```

return render_template('admin/dashboard.html', bookings=bookings)

@app.route('/vehicle/<vehicle_id>')
@login_required
def vehicle_details(vehicle_id):
    vehicle = mongo.db.vehicles.find_one({"_id": ObjectId(vehicle_id)})
    if not vehicle:
        flash("Vehicle not found", "danger")
        return redirect(url_for('index'))
    return render_template('vehicle_details.html', vehicle=vehicle,
today=datetime.now().strftime('%Y-%m-%d'))

@app.route('/book/<vehicle_id>', methods=['POST'])
def book_vehicle(vehicle_id):
    if not is_logged_in():
        flash("Please login to book a vehicle", "info")
        return redirect(url_for('login'))

    start_date = datetime.strptime(request.form.get('start_date'), '%Y-%m-%d')
    end_date = datetime.strptime(request.form.get('end_date'), '%Y-%m-%d')
    user_id = session['user_id']

    if end_date < start_date:
        flash("Return date cannot be before pickup date", "danger")
        return redirect(url_for('vehicle_details', vehicle_id=vehicle_id))

    vehicle = mongo.db.vehicles.find_one({"_id": ObjectId(vehicle_id)})
    if vehicle['available_quantity'] <= 0:
        flash("Vehicle is no longer available", "danger")
        return redirect(url_for('index'))

    # Calculate total price
    days = (end_date - start_date).days or 1
    total_price = days * vehicle['price_per_day']

```

```

# Create booking record
booking = {
    "user_id": ObjectId(user_id),
    "vehicle_id": ObjectId(vehicle_id),
    "start_date": start_date,
    "end_date": end_date,
    "status": "Pending",
    "total_price": total_price,
    "created_at": datetime.now()
}

mongo.db.bookings.insert_one(booking)
# Note: We decrease available_quantity ONLY when Admin APPROVES.
# Or as per user prompt: "On booking -> available_quantity decreases by 1"
mongo.db.vehicles.update_one(
    {"_id": ObjectId(vehicle_id)},
    {"$inc": {"available_quantity": -1}}
)

flash("Booking requested successfully! Waiting for admin approval.",
"success")
return redirect(url_for('dashboard'))

@app.route('/admin/booking/<booking_id>/approve', methods=['POST'])
def approve_booking(booking_id):
    if not is_admin(): return redirect(url_for('index'))
    mongo.db.bookings.update_one({"_id": ObjectId(booking_id)}, {"$set":
{"status": "Approved"}})
    flash("Booking approved!", "success")
    return redirect(url_for('admin_dashboard'))

@app.route('/admin/booking/<booking_id>/cancel', methods=['POST'])
def cancel_booking(booking_id):

```

```

if not is_admin(): return redirect(url_for('index'))
booking = mongo.db.bookings.find_one({"_id": ObjectId(booking_id)})
if booking['status'] != 'Cancelled':
    # Revert availability
    mongo.db.vehicles.update_one({"_id": booking['vehicle_id']}, {"$inc":
{"available_quantity": 1}})
    mongo.db.bookings.update_one({"_id": ObjectId(booking_id)}, {"$set":
{"status": "Cancelled"}})
    flash("Booking cancelled and inventory updated.", "warning")
    return redirect(url_for('admin_dashboard'))

```

```

@app.route('/admin/booking/<booking_id>/complete', methods=['POST'])
def complete_booking(booking_id):
    if not is_admin(): return redirect(url_for('index'))
    booking = mongo.db.bookings.find_one({"_id": ObjectId(booking_id)})
    if booking['status'] == 'Approved':
        # Revert availability
        mongo.db.vehicles.update_one({"_id": booking['vehicle_id']}, {"$inc":
{"available_quantity": 1}})
        mongo.db.bookings.update_one({"_id": ObjectId(booking_id)}, {"$set":
{"status": "Completed"}})
        flash("Rental period completed. Vehicle is now available.", "success")
        return redirect(url_for('admin_dashboard'))

```

```

@app.route('/admin/manage_vehicles')
def manage_vehicles():
    if not is_admin(): return redirect(url_for('index'))
    vehicles = list(mongo.db.vehicles.find())
    return render_template('admin/manage_vehicles.html', vehicles=vehicles)

```

```

@app.route('/admin/vehicle/add', methods=['POST'])
def add_vehicle():
    if not is_admin(): return redirect(url_for('index'))
    name = request.form.get('name')

```

```

v_type = request.form.get('type')
image_url = request.form.get('image_url')
price = int(request.form.get('price_per_day'))
qty = int(request.form.get('total_quantity'))

mongo.db.vehicles.insert_one({
    "name": name,
    "type": v_type,
    "image_url": image_url,
    "price_per_day": price,
    "total_quantity": qty,
    "available_quantity": qty
})
flash("Vehicle added to fleet!", "success")
return redirect(url_for('manage_vehicles'))

@app.route('/admin/vehicle/<vehicle_id>/delete', methods=['POST'])
def delete_vehicle(vehicle_id):
    if not is_admin(): return redirect(url_for('index'))
    mongo.db.vehicles.delete_one({"_id": ObjectId(vehicle_id)})
    flash("Vehicle removed from fleet.", "warning")
    return redirect(url_for('manage_vehicles'))

@app.route('/admin/vehicle/edit', methods=['POST'])
def edit_vehicle():
    if not is_admin(): return redirect(url_for('index'))
    vehicle_id = request.form.get('vehicle_id')
    name = request.form.get('name')
    v_type = request.form.get('type')
    image_url = request.form.get('image_url')
    price = int(request.form.get('price_per_day'))
    new_total = int(request.form.get('total_quantity'))
    old_total = int(request.form.get('old_total_quantity'))
    available = int(request.form.get('available_quantity'))

```

```

quantity_diff = new_total - old_total
new_available = available + quantity_diff
if new_available < 0:
    new_available = 0

mongo.db.vehicles.update_one(
    {"_id": ObjectId(vehicle_id)},
    {"$set": {
        "name": name,
        "type": v_type,
        "image_url": image_url,
        "price_per_day": price,
        "total_quantity": new_total,
        "available_quantity": new_available
    }}
)
flash("Vehicle updated successfully.", "success")
return redirect(url_for('manage_vehicles'))

if __name__ == '__main__':
    app.run(debug=True)

```

6.CONCLUSION

The Smart Vehicle Rental System developed using Java and MongoDB successfully streamlines the management of vehicle rental operations, including vehicle booking, customer management, payment processing, and vehicle tracking. By integrating a secure database with a user-friendly interface, the system improves efficiency, reduces manual errors, and supports real-time data handling. The use of MongoDB provides flexible and scalable data storage for handling customer, vehicle, and booking information efficiently. Overall, this project demonstrates the effective use of Java for application development and MongoDB for reliable database management, resulting in a scalable, efficient, and user-friendly solution for smart vehicle rental administration.

7.REFERENCES

- W3Schools – MongoDB Tutorials
- GeeksforGeeks – MongoDB with Java and MongoDB Driver Tutorials
- Tutorialspoint – MongoDB Java Integration Examples
- MongoDB Inc. – Official MongoDB Documentation and MongoDB Java Driver Guid

